



Name: \_\_\_\_\_ Cohort/Batch: \_\_\_\_\_ Date: \_\_\_\_\_ Score: \_\_\_\_\_

## AMAZON KINESIS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Messaging & Streaming

TOTAL: 10 QUESTIONS

**Q1** What is Amazon Kinesis and what problem in Messaging & Streaming does it solve? Model

---

---

**Q6** How does Amazon Kinesis handle reliability and high availability? Observability

---

---

**Q2** What are the core components or concepts in Amazon Kinesis? Architecture

---

---

**Q7** What observability does Amazon Kinesis provide out of the box? File

---

---

**Q3** How does Amazon Kinesis compare to its main alternatives? Alternatives

---

---

**Q8** What are common performance bottlenecks in Amazon Kinesis? Compliance

---

---

**Q4** How do you get started with Amazon Kinesis for a new project? Hardening

---

---

**Q9** What security best practices apply when running Amazon Kinesis? Testing

---

---

**Q5** What configuration options most affect Amazon Kinesis behavior in production? SSO / IdP

---

---

**Q10** What mistakes do teams commonly make when adopting Amazon Kinesis? Tuning

---

---



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Amazon Kinesis is a tool or platform

Mitigation

Amazon Kinesis is a tool or platform that automates or simplifies a specific concern in the Messaging & Streaming domain, reducing manual effort and operational complexity for engineering teams.

A6 Amazon Kinesis provides HA through leader election, observability

Amazon Kinesis provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 Amazon Kinesis has key building blocks that

Primitives

Amazon Kinesis has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Amazon Kinesis exposes metrics (Prometheus endpoint, structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

Amazon Kinesis exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 Amazon Kinesis trades off simplicity against feature

Hardening

Amazon Kinesis trades off simplicity against feature richness differently than its alternatives. Choose Amazon Kinesis when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install Amazon Kinesis via its package manager

Gotchas

Install Amazon Kinesis via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run Amazon Kinesis with the principle of

Assessment

Run Amazon Kinesis with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

Concurrency

A10 Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.